

FleetMesh 1.8 Product & presentation guide

A working local simulation for SIH PS 26123. Three to six robot processes negotiate jobs and road access. The console lets you inspect decisions, disrupt a run, replay it and review the outcome. Physical AMRs and a general 20% improvement remain future validation.

Start in a few minutes

1. Stop the old terminal with Ctrl+C. Extract this ZIP into a new writable folder.
2. Install Python 3.10 or newer if needed. No API key or third-party Python package is required.
3. Windows: double-click **Start_FleetMesh.bat**. macOS/Linux: run **sh start_fleetmesh.sh**.
4. Keep the terminal open. Use its exact printed URL; an old tab may point to a previous release. The footer must show v1.8.0.
5. Explore the warehouse film, then select **Open console**. Choose **Launch live demo** or **Watch recorded demo** there.

Manual launch: **python start.py --open**. Use python3 or py -3 if that is your Python command. Ctrl+C stops the server. Port busy? The launcher tries the next ten ports, or use --port 9400. Data stays in data/fleetmesh.sqlite3. Use --data-dir PATH for another location.

Workspace	What to do
Overview	Watch the floor. Click a robot or use Inspect robot to see its mission, reason, route and held zones. Use Present or P for the presentation view; Escape exits.
Missions	Search or filter jobs. Add a mission while live or after completion. Ownership transfer applies only before pickup; picked cargo stays with its robot.
Run history	Open a saved experiment. Replay uses recorded frames. Play/pause, scrub and change replay speed. The live experiment continues independently.
Run review	Inspect completion, urgent missions, waits and lead times. Observed checks describe finite saved data. Download the portable HTML report.
Benchmarks	Compare all four policies on the unchanged paired workload. The fixed rule remains the default because it is faster in four of five cohort means.

If something interrupts the presentation

Use Watch recorded demo, then Play replay. It is labelled as an included recording and does not start or replace the live fleet. If a process is stopped, held road zones remain reserved; start a new experiment for a clean fleet. After a server restart, unfinished runs become interrupted records.

Executed on Linux. Windows/macOS launchers are included but untested here. Full visual browser and projector checks remain for your presentation laptop; the available remote browser could not access this local application.

Source requirements: the team-supplied PS 26123 screenshots and official PPT template. This guide describes v1.8. The bundled pitch retains its v1.3 milestone and earlier test counts; the product audit is historical v1.2 evidence. The winner research provides background sources.

The warehouse presentation

A continuous scene, with a contained concept panel. Scroll controls 141 images extracted from the supplied 5.875-second warehouse film. This avoids HTML video decoding and seeking. The overhead illustration now has its own grid cell and caption below the image; it cannot cover the heading. The opening frame remains the fallback.

Entry point	Purpose
/	Warehouse film, shared-space challenge, labelled concept still and saved evidence.
/app	The operations console: live experiments, mission controls, robot inspection, replay and review.
/app#sample	Open the included v1.1 recording without creating a run or replacing live agents.
/app#evidence	Compare all four policies using the retained benchmark results.
/app#review	Read the included recording, outcome and saved-frame checks.

Motion, fallbacks and hardware

Select **Pause motion** to freeze the film. Resume catches up to the current scroll position. Reduced-motion preferences start with a still image and an optional **Enable motion** button. Image failure or a five-second timeout retains the last scene and exposes **Retry motion**. Pause also works during loading. Hidden tabs stop image requests; leaving releases the image cache. The console remains reachable throughout.

The player requests at most four images at once and caches sixteen decoded frames. Obsolete requests are cancelled after rapid scrolling. Narrow layouts fit the whole frame and stack the concept card below its copy. Canvas and WebP support are needed for motion. Device smoothness and responsive layout still need a browser check.

Keep the distinction clear while presenting

The film is AI-generated and illustrative; the overhead panel is explicitly labelled a still image. The concept image does not depict live telemetry. Evidence cards read saved benchmark data. The console reads actual process state; replay is labelled recorded. Starting or replacing a live run requires an explicit console action.

Three-minute acceptance on the presentation laptop

1. Open the printed local URL. Scroll slowly through all four chapters; check for readable text, a visible scene and smooth forward/backward scrubbing.
2. Pause/resume, resize the window, use Tab and Enter, and open the console.
3. Watch the recorded demo, scrub to the end, inspect a robot and open Run review.
4. Launch a fresh live demo; let all seven missions finish, then download its report.
5. Check browser console errors and projector readability. If motion is slow, pause motion or enter /app directly. Keep the labelled recording ready.

Verification here: 38 system/HTTP tests, 34 console integration checks and 35 cinematic checks passed on Linux. The remote browser blocked localhost. Browser rendering, mobile layouts and projector appearance remain unverified.

A short pitch with a verifiable demo

Opening: "Warehouse robots share narrow routes. FleetMesh gives each robot its own planner and peer permissions. We can show what each robot decided, what happened during a disruption, and the recorded result."

Slide	Main point to explain
1 / Problem	PS 26123, BEL. Laptops host the software prototype today. Fill the registered team name and ID before submitting.
2 / Solution	Peers agree on work and road access. The editable floor schematic comes from an actual recorded process state.
3 / Approach	Independent Python processes, direct local UDP, peer auctions, route permissions and local route costs. SQLite and the console observe the result.
4 / Feasibility	The deck retains its v1.3 evidence counts. This release passes 38 system tests plus 34 console and 35 cinematic checks. The 160-run benchmark remains unchanged.
5 / Impact	Target AMR makers and integrators. Both enhanced routing policies exceed 20% in only two of five cohort means. The fixed default beats the learned model in four.
6 / Evidence	Show the live disruption, replay and review. Request a supervised three-AMR validation pilot.

Live walkthrough: pause to explain each stage

Launch live demo and select Present. Set playback to 1x or 2x, then pick Robot 1 in the inspector. Pause when explaining each stage. An urgent seventh job arrives at 5 simulated seconds, an empty aisle closes at 12 s, messages stop at 20 s and return at 28 s, then the aisle reopens at 45 s. Explain waiting and route permissions as they occur. Let all jobs finish, then select Review this run and open the report. Actual wall time depends on the laptop.

The included backup recording completed seven missions in 198.4 simulated seconds, with two route recoveries, one ownership transfer and zero detected collisions. It is a v1.1 capture, not a new performance experiment. Live repetitions may differ.

Answers to likely questions

Where is the edge AI? Each robot can use a small learned delay model locally. It has not beaten our fixed rule consistently, so it stays optional pending independent evaluation.

Is the entire system decentralized? Robot planning and route permissions are peer-based. One laptop still supplies the shared simulation world and clock.

What happens when a robot fails? Its owned zones remain held. Progress may halt. Verified clearance and membership recovery are future work.

Is zero collision a guarantee? No. It is the observed count in finite idealized simulations. Physical sensing, braking and safety integration require testing.

Business fit and the next implementation

Positioning: a local coordination prototype and evaluation workspace for small warehouse fleets. The initial buyer hypothesis is an AMR manufacturer or warehouse integrator; the daily user is a fleet operator or commissioning engineer. Integration and ongoing support are proposed revenue sources. No customer revenue or ROI is claimed.

Existing capability	Our opportunity to validate
MiR Fleet already handles fleet tasks and integration. On-premises deployment exists.	Reduce time to explain a conflict or reproduce a disturbance using inspectable local decisions and a saved run record.
OTTO robots already localize, avoid obstacles and replan.	Demonstrate cross-robot ownership and permissions with a transparent implementation that a partner can evaluate.
Open-RMF coordinates traffic across fleets and continues to evolve.	Test whether a small peer-only coordination module helps a specific three-AMR deployment. Compare against relevant open tooling before claiming an advantage.

These are differentiation hypotheses, not verified missing features in every competitor. Validate with one AMR integrator and one warehouse operator. Ask about current blocking incidents, recovery time, deployment constraints, usable logs and willingness to pilot. Measure integration effort and incident review time before proposing prices.

Present: implemented	Future: required proof
3-6 local OS processes, authenticated loopback UDP, mission auctions and conservative zone access.	Three physical edge computers; real network delay/loss; safe membership and restart procedures.
Ideal grid simulation, aisle changes, peer-message interruption, collision observation and route recovery.	ROS 2/Nav2 or equivalent integration; noisy localization; braking, watchdogs and supervised three-AMR trials.
Operator UI, presentation view, robot inspection, replay, run review, SQLite and exports.	Presentation-laptop visual testing, operator usability trials and deployment-specific security controls.
Fixed and learned routing policies with a reproducible development benchmark.	Frozen independent workloads, stronger baselines, delivery-only and dock-return metrics, paired uncertainty and direct AI-vs-fixed comparison.

Plan to the 16 September internal round

13 September: freeze the release, assign demo and Q&A owners, then test the actual browser and projector.

14 September: rehearse interruptions and explanations; collect a labelled backup recording.

15 September: enter team details, review the six-slide PDF, check links and back up the package.

16 September: submit by your college deadline. Keep the product and recorded evidence available offline.

Evidence, limits and submission checks

This release implements the local software simulation and cinematic frontend. Browser animation and layout acceptance remain unverified here. Competition selection and industrial readiness remain unproven. The highest-value next work is external validation of the coordination method and its performance, followed by safe physical integration.

Evidence	What it establishes
38 system tests	Protocol behaviors, persistence, validation, process operation and truthful run-review summaries pass on Linux.
34 console + 35 cinematic checks	Console checks use a real HTTP/UDP backend. Scene checks exercise real player/page code with image, canvas and DOM stubs. They do not verify browser layout or device smoothness.
160 development benchmark runs	Five cohorts x eight paired seeds x four policies. All completed with zero detected collisions. This benchmark is retained unchanged from the earlier engine version.
Six process audit cases / 81 jobs	Three, four and six workers complete the predeclared functional workloads. This is retained v1.2 evidence, separate from the speed comparison.
Included recorded demo	201 recorded frames show one seven-job v1.1 demonstration. The replay and review label the source and remain read-only.

Reproduce or inspect

```
python -m unittest discover -s tests -v
node tests/ui_smoke.cjs
node --experimental-vm-modules tests/cinematic_checks.mjs
(Node is optional for these developer checks; not needed to run FleetMesh)
python tools/audit_runtime.py
python evaluate.py (regenerates the model and overwrites benchmark files)
```

Inspect evidence/tests.txt, cinematic_checks.json, sequence_asset.json, ui_http_checks.json, benchmark.csv and summary.json. Run review checks saved-frame zone ownership and active-job assignments, not every controller tick. Its collision count comes from the simulator. Lead times begin at each job's submission, including queueing.

Before submission

Replace all [Team name], [Enter registered ID] and [Enter registered team name] placeholders. Keep the six required slides. The supplied template calls for PDF submission; follow your college instructions if they differ. Confirm titles, chart labels and the final page on your own display. Speaker notes include the complete explanation and sources. Never replace measured numbers with aspirational targets.

References and further reading

[MiR Fleet: product capabilities](#)

[OTTO: onboard autonomy](#)

[Open-RMF: next-generation traffic management \(2026\)](#)